VECTOR

MICROSAR CRC

Technical Reference

Version 5.00.00

Authors	Michael Goß, Leon Jonas
Status	Released

Document Information

History

Author	Date	Version	Remarks
Tobias Schmid	2006-12-13	1.0	Initial Version
Tobias Schmid	2008-01-21	3.00.00	Update to ASR 2.1 Changed versioning to new notation
Claudia Mausz	2008-05-19	4.00.00	Update to ASR 3 Add Crc8 calculation
Michael Goß	2014-11-18	4.01.00	Update to ASR 4 Add Crc8H2F calculation
Michael Goß	2015-05-08	4.02.00	SafeBSW Add Crc32P4 calculation
Michael Goß	2016-11-24	4.03.00	Add Crc64 calculation
Leon Jonas	2020-06-02 2020-06-08	5.00.00	Add slicing-by-4 feature Add slicing-by-8 feature

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_CRCLibrary.pdf	V4.2.0
[2]	AUTOSAR	AUTOSAR_SWS_CRCLibrary.pdf	V4.3.0
[3]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	V1.6.0
[4]	IEEE	Frank L. Berry, Michael E. Kounavis, "Novel Table Lookup-Based Algorithms for High-Performance CRC Generation", IEEE Transactions on Computers, vol. 57, no. , pp. 1550-1560, November 2008, doi:10.1109/TC.2008.85	-

Scope of the Document

This technical reference describes the general use of the CRC library basis software. There are no aspects which are controller specific.



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6

2 Introduction 7
2.1 Architecture Overview 8

3 Functional Description 9
3.1 Features 9
3.1.1 Deviations 9
3.1.2 Additions/ Extensions 10
3.1.3 Slicing-by-x 10
3.2 Initialization 10
3.3 States 10
3.4 Main Functions 10
3.5 Error Handling 10
3.5.1 Development Error Reporting 10
3.5.2 Production Code Error Reporting 10
3.5.3 Parameter Checking 10

4 Integration 11
4.1 Scope of Delivery 11
4.1.1 Static Files 11
4.1.2 Dynamic Files 11
4.2 Include Structure 11

5 API Description 12
5.1 Type Definitions 12
5.2 Interrupt Service Routines provided by CRC 12
5.3 Services provided by CRC 12
5.3.1 Crc_CalculateCRC8 12
5.3.2 Crc_CalculateCRC8H2F 13
5.3.3 Crc_CalculateCRC16 14
5.3.4 Crc_CalculateCRC32 15
5.3.5 Crc_CalculateCRC32P4 17
5.3.6 Crc_CalculateCRC64 18
5.3.7 Crc_GetVersionInfo 20
5.4 Services used by CRC 21
5.5 Callback Functions 22
5.6 Configurable Interfaces 22
5.6.1 Notifications 22

5.6.2 Callout Functions 22
5.6.3 Hook Functions 22

6 Configuration 23
6.1 Configuration Variants 23

7 Glossary and Abbreviations 24
7.1 Glossary 24
7.2 Abbreviations 24

8 Contact 25

Illustrations

Figure 2-1 AUTOSAR 4.x Architecture Overview 8
Figure 4-1 Include structure 11

Tables

Table 1-1 Component history 6
Table 3-1 Supported AUTOSAR standard conform features 9
Table 3-2 Not supported AUTOSAR standard conform features 9
Table 3-3 Features provided beyond the AUTOSAR standard 10
Table 4-1 Static files 11
Table 4-2 Generated files 11
Table 5-1 Type definitions 12
Table 5-2 Std_VersionInfoType 12
Table 5-3 SAE-J1850 CRC8 Standard 13
Table 5-4 Crc_CalculateCRC8 13
Table 5-5 CRC calculation based on 0x2F polynomial 14
Table 5-6 Crc_CalculateCRC8H2F 14
Table 5-7 CCITT CRC16 Standard 15
Table 5-8 Crc_CalculateCRC16 15
Table 5-9 IEEE-802.3 CRC32 Ethernet Standard 16
Table 5-10 Crc_CalculateCRC32 17
Table 5-11 CRC32 calculation based on 0xF4ACFB13 polynomial (E2E Protection Profile 4) 17
Table 5-12 Crc_CalculateCRC32P4 18
Table 5-13 IEEE-802.3 CRC32 Ethernet Standard 18
Table 5-14 Crc_CalculateCRC64 19
Table 5-15 Crc_GetVersionInfo 21
Table 7-1 Glossary 24
Table 7-2 Abbreviations 24

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
4.00.00	Update to AUTOSAR Version 3 Add Crc8 calculation
4.01.00	Update to AUTOSAR Version 4 Add Crc8 calculation based on 0x2F polynomial
4.02.00	Crc32 E2E Profile 4 routine was added due to SafeBSW
4.03.00	Crc64 (ECMA) E2E Profile 7 routine was added
5.00.00	Slicing-by-4 feature was added Slicing-by-8 feature was added

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module CRC as specified in [1].

Supported AUTOSAR Release*:	4	
Supported Configuration Variants:	pre-compile	
Vendor ID:	CRC_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	CRC_MODULE_ID	201 decimal (according to ref. [3])

* For the precise AUTOSAR Release 4.x please see the release specific documentation.

This component implements service functions in ANSI C for calculating CRC checksums. The module allows pre-compile configuration of the calculation method, which is used to compute the CRC values. The six possible methods are table based or runtime calculation of CRC values.

There are six different CRC calculation services available:

- Two different services for checksum calculation of 8bit CRC value from a buffer
- Checksum calculation of 16bit CRC value from a buffer
- Two different services for checksum calculation of 32bit CRC value from a buffer
- Checksum calculation of 64bit CRC value from a buffer

2.1 Architecture Overview

The following figure shows where the CRC is located in the AUTOSAR architecture.

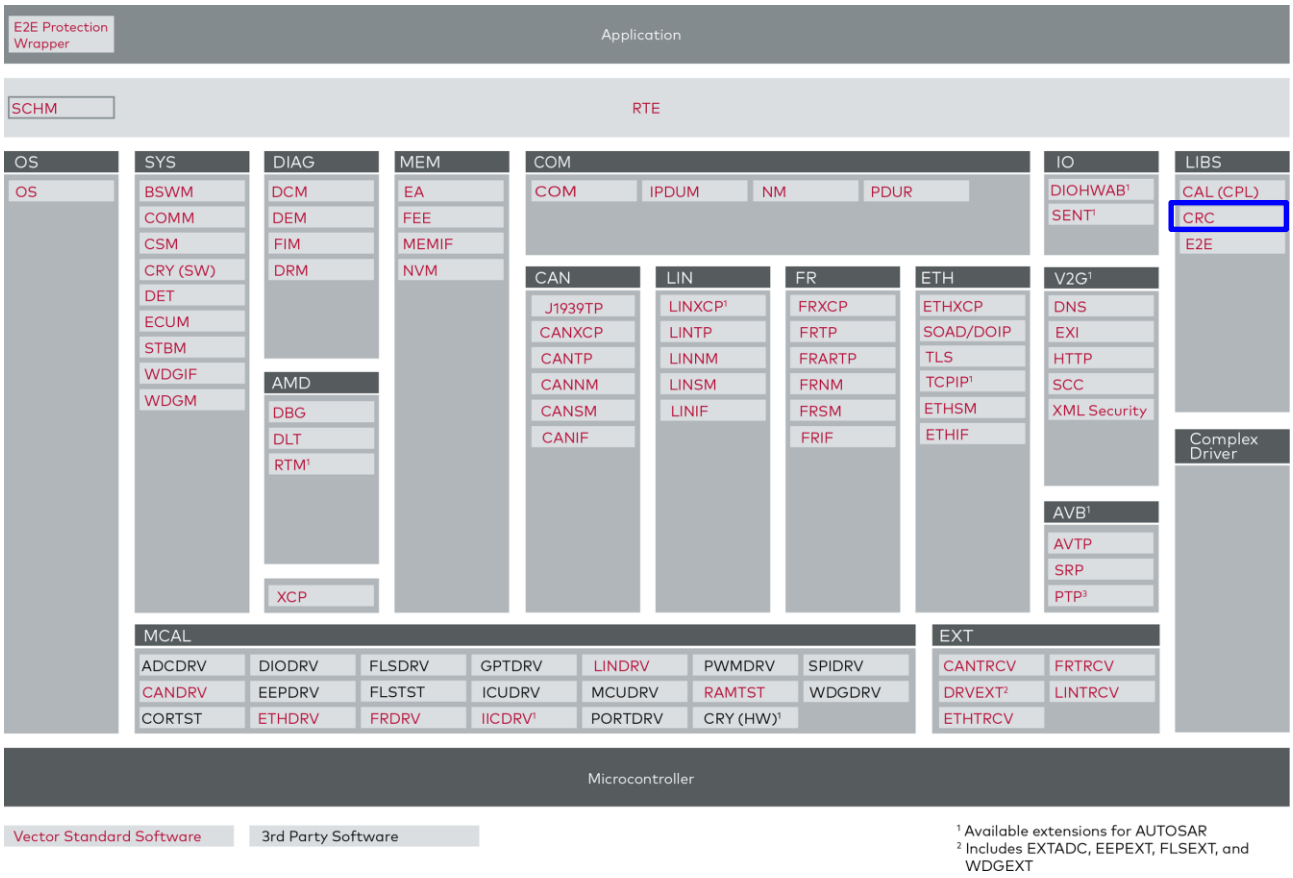


Figure 2-1 AUTOSAR 4.x Architecture Overview

No interface from other module is required. The CRC component is a service module. It can be called from any module or application (e.g. NVRAM Manager).

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the CRC.

The AUTOSAR standard functionality is specified in [1], the corresponding features are listed in the tables

> Table 3-1 Supported AUTOSAR standard conform features

> Table 3-2 Not supported AUTOSAR standard conform features

Vector Informatik provides further CRC functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-3 Features provided beyond the AUTOSAR standard

The following features specified in [1] are supported:

Supported AUTOSAR Standard Conform Features
Crc8 calculation based on the SAE-J1850 CRC8 Standard
Crc8 calculation based on 0x2F polynomial
Crc16 calculation based on the CCITT CRC16 Standard
Crc32 calculation based on the IEEE-802.3 Ethernet Standard
Crc32 calculation based on 0xF4ACFB13 polynomial (E2E Profile 4)
Crc64 calculation based on 0x42F0E1EBA9EA3693 polynomial (E2E Profile 7)
For all CRC calculations following methods can be used: > Table based calculation: results in fast execution, but increases code size (ROM table) > Runtime calculation: results in smaller code size, but slower execution time
All routines are re-entrant and can be used by multiple applications at the same time.

Table 3-1 Supported AUTOSAR standard conform features

3.1.1 Deviations

The following features specified in [1] are not supported:

Not Supported AUTOSAR Standard Conform Features
Hardware based CRC calculation
Debugging concept

Table 3-2 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Slicing-by-4 and slicing-by-8 for the following Crc calculations: > Crc16 based on the CCITT CRC16 Standard > Crc32 based on 0xF4ACFB13 polynomial (E2E Profile 4) > Crc64 based on 0x42F0E1EBA9EA3693 polynomial (E2E Profile 7)

Table 3-3 Features provided beyond the AUTOSAR standard

3.1.3 Slicing-by-x

The slicing-by-x feature uses x number of precalculated tables instead of a single table. This results in an increase of the needed amount of ROM. However, since the values of the tables don't have to be calculated at runtime anymore and x tables can be used instead of one, there are performance gains.

The currently supported values for x are 4 and 8 for the profiles specified in Table 3-3.

Further information about the exact ramifications of this feature can be found in the paper referenced in referenced document number [4].

3.2 Initialization

No initialization is necessary.

3.3 States

This component does not contain any state machine.

3.4 Main Functions

This component does not have a main function. It only contains the routines to calculate the checksums. These API functions are called from other modules, e.g. NVRAM Manager.

3.5 Error Handling

3.5.1 Development Error Reporting

Module CRC does not provide any development error detection mechanism.

3.5.2 Production Code Error Reporting

Module CRC does not provide any production error detection mechanism.

3.5.3 Parameter Checking

Module CRC's functions do not perform any parameter checks.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR CRC into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the CRC contains the files which are described in the chapters 4.1.1 and 4.1.2.

4.1.1 Static Files

File Name	Description
Crc.c	Implementation of the CRC routines
Crc.h	Header file of the CRC routines

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the configuration tool DaVinci Configurator.

File Name	Description
Crc_Cfg.h	Configuration of the CRC routines

Table 4-2 Generated files

4.2 Include Structure

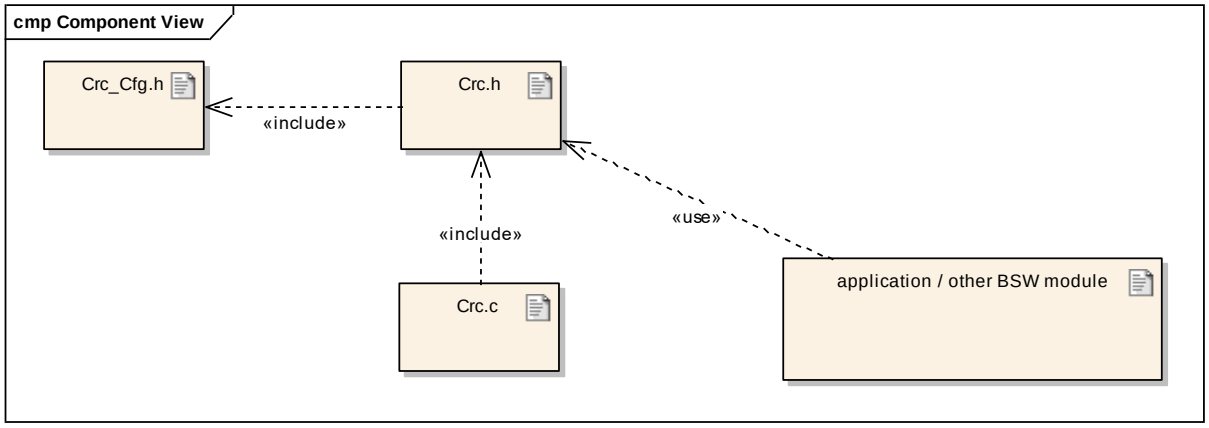


Figure 4-1 Include structure

5 API Description

5.1 Type Definitions

The types defined by the CRC are described in this chapter.

Type Name	C-Type	Description	Value Range
Crc_DataRefType	P2CONST	Pointer type for pointers to data buffers, from which the CRC value is to be calculated.	`0-255 (uint8)`

Table 5-1 Type definitions

Std_VersionInfoType

Struct Element Name	C-Type	Description	Value Range
Std_VersionInfoType	Struct	Returns the version information	`uint16 vendorID`
			`uint16 moduleID`
			`uint8 sw_major_version`
			`uint8 sw_minor_version`
			`uint8 sw_patch_version`

Table 5-2 Std_VersionInfoType

5.2 Interrupt Service Routines provided by CRC

The CRC component does not provide any interrupt service routines.

5.3 Services provided by CRC

5.3.1 Crc_CalculateCRC8

Prototype	
`uint8` **`Crc_CalculateCRC8`** `(Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8, boolean Crc_IsFirstCall )`	
Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue8`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue8. FALSE: Subsequent call in a call sequence; Crc_StartValue8 is interpreted to be the return value of the previous function call.
Return code	
`uint8`	8 bit result of CRC calculation

Functional Description
This service performs a CRC8 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue8. The CRC8 routine is based on the SAE-J1850 CRC8 Standard:

SAE-J1850 CRC8 Standard	value
CRC result width	8 bits
Polynomial	1Dh
Initial value	FFh
Input data reflected	No
Result data reflected	No
XOR value	FFh
Check	4Bh
Magic check	C4h

Table 5-3 SAE-J1850 CRC8 Standard

Particularities and Limitations
> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time.
Expected Caller Context
> There is no specific caller. CRC can be called from anywhere.

Table 5-4 Crc_CalculateCRC8

5.3.2 Crc_CalculateCRC8H2F

Prototype
`uint8 Crc_CalculateCRC8H2F (Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint8 Crc_StartValue8H2F, boolean Crc_IsFirstCall )`

Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue8H2F`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue8H2F. FALSE: Subsequent call in a call sequence; Crc_StartValue8H2F is interpreted to be the return value of the previous function call.
Return code	
`uint8`	8 bit result of CRC calculation

Functional Description
This service performs a CRC8 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue8H2F. The CRC8 routine is based on the generator polynomial 0x2F:

CRC8 routine based on 0x2F polynomial	value
CRC result width	8 bits
Polynomial	2Fh
Initial value	FFh
Input data reflected	No
Result data reflected	No
XOR value	FFh
Check	DFh
Magic check	42h

Table 5-5 CRC calculation based on 0x2F polynomial

Particularities and Limitations
> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time.
Expected Caller Context
> There is no specific caller. CRC can be called from anywhere.

Table 5-6 Crc_CalculateCRC8H2F

5.3.3 Crc_CalculateCRC16

Prototype
`uint16 Crc_CalculateCRC16 (Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint16 Crc_StartValue16, boolean Crc_IsFirstCall )`

Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue16`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue16. FALSE: Subsequent call in a call sequence; Crc_StartValue16 is interpreted to be the return value of the previous function call.
Return code	
`uint16`	16 bit result of CRC calculation

Functional Description
This service performs a CRC16 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue16. The CRC16 routine is based on the CCITT CRC16 Standard:

CCITT CRC16 Standard	value
CRC result width	16 bits
Polynomial	1021h
Initial value	FFFFh
Input data reflected	No
Result data reflected	No
XOR value	0000h
Check	29B1h
Magic check	0000h

Table 5-7 CCITT CRC16 Standard

Particularities and Limitations
> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time. > If the single table doesn't provide enough speed, the slicing-by-4 feature can be enabled to further increase the performance. This comes at the cost of an additional increase in ROM size.
Expected Caller Context
> There is no specific caller. CRC can be called from anywhere.

Table 5-8 Crc_CalculateCRC16

5.3.4 Crc_CalculateCRC32

Prototype
`uint32 Crc_CalculateCRC32 (Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)`

Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue32`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue32. FALSE: Subsequent call in a call sequence; Crc_StartValue32 is interpreted to be the return value of the previous function call.

Return code	
uint32	32 bit result of CRC calculation

Functional Description

This service performs a CRC32 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue32. The CRC32 routine is based on the IEEE-802.3 CRC32 Ethernet Standard:

IEEE-802.3 CRC32 Ethernet Standard	value
CRC result width	32 bits
Polynomial	04C11DB7h
Initial value	FFFFFFFFh
Input data reflected	Yes
Result data reflected	Yes
XOR value	FFFFFFFFh
Check	CBF43926h
Magic check*	DEBB20E3h

Table 5-9 IEEE-802.3 CRC32 Ethernet Standard



***Important note**
To match the magic check value, the CRC must be appended in little endian format, i.e. low significant byte first. This is due to the reflections of the input and the result.

Example of magic check: calculation of IEEE-802.3 CRC32 over data bytes 31h 32h 33h 34h 35h 36h 37h 38h 39h:

> CRC generation: CRC over 31h 32h 33h 34h 35h 36h 37h 38h 39h, initial value FFFFFFFFh:
>
> > CRC-result: CBh F4h 39h 26h (See check value)
>
> CRC check: CRC over 31h 32h 33h 34h 35h 36h 37h 38h 39h 26h 39h F4h CBh (CRC-result was appended to data bytes in little endian format), initial value FFFFFFFFh:
>
> > CRC-result: 21h 44h DFh 1Ch
> >
> > Magic check = CRC-result XORed with ‘XOR value’:
> >
> > DEBB20E3h = 2144DF1Ch xor FFFFFFFFh

Particularities and Limitations

> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time.

Expected Caller Context
> There is no specific caller. CRC can be called from anywhere.

Table 5-10 Crc_CalculateCRC32

5.3.5 Crc_CalculateCRC32P4

Prototype	
`uint32 Crc_CalculateCRC32P4 (Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint32 Crc_StartValue32, boolean Crc_IsFirstCall)`	
Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue32`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue32. FALSE: Subsequent call in a call sequence; Crc_StartValue32 is interpreted to be the return value of the previous function call.
Return code	
`uint32`	32 bit result of CRC calculation
Functional Description	
This service performs a CRC32 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue32. The CRC32 routine is based on the 0xF4ACFB13 polynomial (E2E Protection Profile 4):	

CRC 32 routine based on 0xF4ACFB13 polyonmial	value
CRC result width	32 bits
Polynomial	F4ACFB13h
Initial value	FFFFFFFFh
Input data reflected	Yes
Result data reflected	Yes
XOR value	FFFFFFFFh
Check	1697D06Ah
Magic check	6FB32240h

Table 5-11 CRC32 calculation based on 0xF4ACFB13 polynomial (E2E Protection Profile 4)

Particularities and Limitations
> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time. > If the single table doesn't provide enough speed, the slicing-by-4 feature can be enabled to further increase the performance. This comes at the cost of an additional increase in ROM size.
Call context
> There is no specific caller. CRC can be called from anywhere.

Table 5-12 Crc_CalculateCRC32P4

5.3.6 Crc_CalculateCRC64

Prototype	
`uint64 Crc_CalculateCRC64 (Crc_DataRefType Crc_DataPtr, uint32 Crc_Length, uint64 Crc_StartValue64, boolean Crc_IsFirstCall)`	
Parameter	
`Crc_DataPtr`	Pointer to start address of data block to be calculated
`Crc_Length`	Length of data to be calculated (in bytes)
`Crc_StartValue64`	Value the algorithm starts with
`Crc_IsFirstCall`	TRUE: First call in a sequence or individual CRC calculation; start from initial value, ignore Crc_StartValue64. FALSE: Subsequent call in a call sequence; Crc_StartValue64 is interpreted to be the return value of the previous function call.
Return code	
uint64	64 bit result of CRC calculation

Functional Description

This service performs a CRC64 calculation synchronously on Crc_Length data bytes, pointed to by Crc_DataPtr, with the starting value of Crc_StartValue64. The CRC64 routine is based on the ECMA Standard used by E2E Profile 7:

ECMA Standard	value
CRC result width	64 bits
Polynomial	42F0E1EBA9EA3693h
Initial value	FFFFFFFFFFFFFFFFh
Input data reflected	Yes
Result data reflected	Yes
XOR value	FFFFFFFFFFFFFFFFh
Check	995DC9BBDF1939FAh
Magic check*	49958C9ABD7D353Fh

Table 5-13 IEEE-802.3 CRC32 Ethernet Standard

***Important note**
To match the magic check value, the CRC must be appended in little endian format, i.e. low significant byte first. This is due to the reflections of the input and the result.

Example of magic check: calculation of ECMA CRC64 over data bytes 31h 32h 33h 34h 35h 36h 37h 38h 39h:

> CRC generation: CRC over 31h 32h 33h 34h 35h 36h 37h 38h 39h, initial value FFFFFFFFFFFFFFFFh:
>
> > CRC-result: 99h 5Dh C9h BBh DFh19h 39h FAh (See check value)
>
> CRC check: CRC over 31h 32h 33h 34h 35h 36h 37h 38h 39h FAh 39h 19h DFh BBh C9h 5Dh 99h (CRC-result was appended to data bytes in little endian format), initial value FFFFFFFFFFFFFFFFh:
>
> > CRC-result: B6h 6Ah 73h 65h 42h 82h CAh C0h
>
> > Magic check = CRC-result XORed with 'XOR value':
> >
> > 49958C9ABD7D353Fh = B66A73654282CAC0h XOR FFFFFFFFFFFFFFFFh

Particularities and Limitations

> If large data blocks have to be calculated (>32 bytes), the table based calculation method should be configured in order to decrease the calculation time.
> If the single table doesn't provide enough speed, the slicing-by-4 feature can be enabled to further increase the performance. This comes at the cost of an additional increase in ROM size.

Call context

> There is no specific caller. CRC can be called from anywhere.

Table 5-14 Crc_CalculateCRC64

5.3.7 Crc_GetVersionInfo

Prototype
`void Crc_GetVersionInfo ( Std_VersionInfoType *Versioninfo )`

Parameter	
`Versioninfo`	Pointer to where to store the version information of this module

Return code	
None	--
Functional Description	
This service returns the version information of the module. The version information includes: > Module ID > Vendor ID > Vendor specific version numbers This function is pre-compile time configurable On/Off by the configuration parameter CrcVersionInfoApi.	
Particularities and Limitations	
> None	
Expected Caller Context	
> Application	

Table 5-15 Crc_GetVersionInfo

5.4 Services used by CRC

The CRC component does not use services from other modules.

5.5 Callback Functions

Module CRC's functions do not provide any callback functions.

5.6 Configurable Interfaces

5.6.1 Notifications

The CRC module does not provide notifications for upper layers. The CRC routines return the checksum; this value will be evaluated by application or other modules.

5.6.2 Callout Functions

The CRC module does not provide callout functions.

5.6.3 Hook Functions

The CRC module does not provide hook functions.

6 Configuration

6.1 Configuration Variants

The CRC supports the configuration variants

> VARIANT-PRE-COMPILE

The configuration classes of the CRC parameters depend on the supported configuration variants. For their definitions please see the Crc_bswmd.arxml file.

CRC can be configured using the following tool:

> DaVinci Configurator 5 (AUTOSAR 4 packages only). Parameters are explained within the tool.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
CRC	Cyclic Redundancy Check

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CCITT	Commité Consultatif Internationale Télégrafique et Téléfonique
DEM	Diagnostic Event Manager
DET	Development Error Tracer
E2E	End to End
ECMA	European Computer Manufacturers Association
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
IEEE	Institute of Electrical and Electronics Engineers
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
NVRAM Manager	Non Volatile Random Access Memory Manager
ROM	Read Only Memory
RTE	Runtime Environment
SAE	Society of Automotive Engineers
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com

© 2020 Vector Informatik GmbH